CHAPTER 7

Lambdas and Functional Interfaces

Java Refresher · Depth 9/10 · Java 17+ · Allman-style runnable code

You will learn to read and write functional-interface callbacks, predict how captures and method references behave, and avoid the common traps seen in Java and Spring applications.

Includes 22 focused lessons, a glossy infographic, six consolidated Java programs, explanations, expected outputs, and practice questions.

JAVA CHAPTER 7 • LAMBDAS & FUNCTIONAL INTERFACES

Syntax • target typing • built-in interfaces • method references • capture • pitfalls

1 LAMBDA SYNTAX

(a, b) -> a + b

- (x) -> x * 2 is an expression body.
- Block bodies need return when returning a value.
- A lambda needs a functional-interface target.

2 FUNCTIONAL INTERFACE

Single Abstract Method (SAM)

- Exactly one abstract method.
- @FunctionalInterface is a compiler check.
- Default and static methods are permitted.

3 THE FOUR CORE TYPES

test · apply · accept · get

- Predicate<T> → boolean
- Function<T,R> → R
- Consumer<T> → void
- Supplier<T> → T

4 COMPOSITION

Build small behavior from pieces

- Predicate: and / or / negate.
- Function: compose / andThen.
- Consumers: andThen for ordered effects.

5 METHOD REFERENCES

Use when simpler than a lambda

- Type::staticMethod
- obj::instanceMethod
- Type::instanceMethod
- Type::new constructor reference

6 CAPTURE & SCOPE

Beware shared mutable state

- Captured locals must be effectively final.
- this means the enclosing instance.
- Mutable objects can still be mutated.

7 OPERATORS & SPECIALIZATIONS

Choose semantic types

- UnaryOperator<T> and BinaryOperator<T>.
- IntPredicate, IntUnaryOperator, etc.
- Primitive forms can avoid boxing.

8 EXCEPTIONS & TIMING

Definition is not execution

- Checked exceptions obey the SAM throws clause.
- Lambda body runs when invoked.
- Keep causes when translating failures.

9 BEST PRACTICES

Clarity over cleverness

- Prefer short, readable lambdas.
- Do not hide heavy side effects.
- Use named methods for complex behavior.
- Streams and Optional come next chapter.

Learning roadmap

Part I — Syntax and contracts (Lessons 1–5). Part II — The standard interfaces (Lessons 6–11). Part III — Method references, captures and scope (Lessons 12–16). Part IV — Applied behavior and pitfalls (Lessons 17–22).

How to study

Read the rule, predict output before revealing it, then trace the invocation. In IntelliJ, run the matching topic demo, modify one input, and watch which lambda actually executes. Examples are teaching snippets within a surrounding class/method unless shown as complete declarations.

LESSON 01 OF 22

What a lambda actually is

THE CORE RULE

A lambda supplies behavior for the single abstract method of a target functional interface; it does not create a free-standing function.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Runnable r = () -> System.out.println("Go");
r.run();
```

EXPECTED OUTPUT / RESULT

```
Go
```

WHY THIS WORKS — DEEP EXPLANATION

Java is nominally typed: lambda expressions gain meaning from an assignment, argument, or cast that identifies the target interface. They are instances of functional-interface types rather than independent Java functions. The runtime may use invokedynamic; do not assume a new anonymous object is allocated for every evaluation.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Contrast with an anonymous class: Runnable r = new Runnable() { public void run() { ... } }; the lambda keeps the callback contract but uses lexical scoping. Do not compare lambda object identity or generated classes in business logic. Note: calling r.run() on the caller thread is not the same as starting a new Thread(r).

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: Go.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Try declaring a lambda without assigning it to a target type. Why can the compiler not infer a stand-alone type?

LESSON 02 OF 22

Single-parameter and multi-parameter syntax

THE CORE RULE

Parameter parentheses are optional only for one inferred parameter; the arrow separates parameters from the expression or block.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Function<String,Integer> size = s -> s.length();
System.out.println(size.apply("Java"));
```

EXPECTED OUTPUT / RESULT

```
4
```

WHY THIS WORKS — DEEP EXPLANATION

Inferred parameter types are governed by the functional interface method. You can declare explicit parameter types, but do it consistently in a parameter list. For two or more parameters, parentheses are required. Java supports var in lambda parameters when used consistently across the entire parameter list.

PRODUCTION DETAIL / SECOND-LAYER REASONING

An inferred lambda parameter cannot mix an explicit type with an inferred type in the same parameter list. For example, (String a, b) is not valid, but (String a, String b) is. A multiline lambda with a block can declare local values, use if/else, and throw exceptions permitted by its target method.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 4.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Rewrite s -> s.length() with an explicitly typed parameter.

LESSON 03 OF 22

Expression bodies versus block bodies

THE CORE RULE

Expression lambdas yield an expression value where the target method returns a value; blocks require explicit return statements on every path.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
BinaryOperator<Integer> add = (a, b) ->
{
int sum = a + b;
return sum;
};
System.out.println(add.apply(3, 4));
```

EXPECTED OUTPUT / RESULT

```
7
```

WHY THIS WORKS — DEEP EXPLANATION

An expression body has no braces and no return keyword. A statement block is like an ordinary method body: local variables, branches, and return are allowed. A void-compatible expression body must be a statement expression. Allman braces are used in the accompanying Java files.

PRODUCTION DETAIL / SECOND-LAYER REASONING

For a block implementing Function<T,R>, every reachable normal return path must return R. For a Consumer<T>, return; is permitted but return aValue; is not. Some void-compatible expressions (such as a method invocation) can form expression bodies; arithmetic expressions cannot stand alone as void statements.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 7.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

What happens if the return in a value-producing block is omitted?

LESSON 04 OF 22

Functional interface contract and @FunctionalInterface

THE CORE RULE

A functional interface has exactly one abstract method after ignoring methods from Object; it may also declare defaults and static methods.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
@FunctionalInterface
interface Checker
{
boolean test(String s);
}
```

EXPECTED OUTPUT / RESULT

```
Compiles
```

WHY THIS WORKS — DEEP EXPLANATION

The annotation is a compiler check, not a prerequisite for lambda compatibility. Object-equivalent signatures such as toString() do not count against the abstract-method total. Inherited abstract methods with matching signatures can collapse into one functional contract. Adding a second unrelated abstract method breaks lambda compatibility.

PRODUCTION DETAIL / SECOND-LAYER REASONING

The annotation checks a structural language rule, but Java still treats interfaces nominally: two interfaces with identical SAM signatures remain distinct types. Default methods provide reusable behavior, while static methods belong to the interface name. Do not add extra abstract operations casually to published callback APIs.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: Compiles.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Add a default method and then add a second abstract method; predict which change fails.

LESSON 05 OF 22

Target typing in variables, parameters and returns

THE CORE RULE

The same lambda syntax can target multiple interfaces; the surrounding context determines parameter and return types.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Predicate<String> longName = s -> s.length() > 3;
System.out.println(longName.test("Java"));
```

EXPECTED OUTPUT / RESULT

```
true
```

WHY THIS WORKS — DEEP EXPLANATION

A compiler resolves the target interface for lambdas passed into methods, returned from methods, and stored in fields or variables. When overloaded methods accept unrelated functional interfaces, a lambda can be ambiguous and need a cast or a more explicit design. Choose descriptive interface types rather than needless casts.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Overload resolution uses target types and the shape of the lambda body. Some overloads remain inherently ambiguous. For example overloaded use(Callable<String>) and use(Supplier<String>) may require an explicit target cast. Usually different method names make a cleaner API than forcing the caller to cast lambdas.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: true.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Why can two overloads taking unrelated functional interfaces make a call ambiguous?

LESSON 06 OF 22

Predicate and boolean composition

THE CORE RULE

Predicate<T> consumes T and returns boolean; and, or, and negate compose tests with short-circuit semantics.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Predicate<String> p = s -> s.startsWith("J");
System.out.println(p.and(s -> s.length() == 4).test("Java"));
```

EXPECTED OUTPUT / RESULT

```
true
```

WHY THIS WORKS — DEEP EXPLANATION

The test operation is useful in validation, filtering, authorization checks, and collection searches. Composition evaluates left-to-right; if the left test of and is false, the right test is skipped. Predicates should usually be side-effect-free because evaluation count and order can matter.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Predicate.and() and or() short-circuit like Java && and ||. For expensive checks, place the cheap null or validity guard first. Always be careful with predicate order: s -> s.length()>3 throws on null unless a previous predicate rules null out.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: true.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

How does p.negate() differ from changing the input?

LESSON 07 OF 22

Function<T,R> and function composition

THE CORE RULE

Function transforms one value into another; compose executes the supplied function first, and andThen executes it second.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Function<String,String> trim = String::trim;
Function<String,Integer> length = String::length;
System.out.println(trim.andThen(length).apply(" Java "));
```

EXPECTED OUTPUT / RESULT

```
4
```

WHY THIS WORKS — DEEP EXPLANATION

Think in data-flow order. f.andThen(g) means g(f(x)); f.compose(g) means f(g(x)). Different input/output type parameters make a useful compile-time contract. Functions often encapsulate mapping logic; use a named method instead when a transformation becomes too complex.

PRODUCTION DETAIL / SECOND-LAYER REASONING

An easy diagram is input → trim → String result → length → Integer result. Function.identity() returns its input; use it when a mapping API requires a Function but no transformation is needed. Pay attention to compose and andThen parameter types: a chain must connect the upstream output to the downstream input.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 4.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Could you reverse trim.andThen(length) without changing types? Explain.

LESSON 08 OF 22

Consumer<T> and side-effect boundaries

THE CORE RULE

Consumer accepts a value and returns void; it is intended for deliberate effects such as logging or writing output.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Consumer<String> log = s -> System.out.println(s);
log.accept("event");
```

EXPECTED OUTPUT / RESULT

```
event
```

WHY THIS WORKS — DEEP EXPLANATION

Use andThen to sequence two consumers, understanding that an exception in the first prevents the second from running. Avoid hiding important mutations in consumers used deep in data processing. A Consumer is not a Function because it has no return value.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Consumer.andThen(second) composes effects in order, but returns no undo operation. For telemetry or notifications, failures may interrupt the sequence. When correctness needs transactions or retries, do not pretend a chain of Consumers provides those guarantees.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: event.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Predict the behavior if the first consumer throws before andThen executes.

LESSON 09 OF 22

Supplier<T> and deferred work

THE CORE RULE

Supplier has no arguments and produces a value when get() is invoked; each call may compute or retrieve a new result.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Supplier<String> s = () -> "ready";
System.out.println(s.get());
```

EXPECTED OUTPUT / RESULT

```
ready
```

WHY THIS WORKS — DEEP EXPLANATION

A supplier is useful for factories, deferred expensive messages, lazy defaults, and dependency boundaries. It does not itself cache or memoize: repeated get() calls invoke the supplied logic unless your implementation explicitly remembers a value. Beware suppliers that introduce surprising side effects.

PRODUCTION DETAIL / SECOND-LAYER REASONING

A supplier delays a calculation, but only the receiving API determines when or how often it is invoked. For example Optional.orElse(expensive()) eagerly evaluates the argument; Optional.orElseGet(() -> expensive()) can defer the work until the Optional is empty. This difference is especially useful for expensive defaults.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: ready.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Does Supplier guarantee the same returned object on every get()?

LESSON 10 OF 22

UnaryOperator, BinaryOperator and BiFunction

THE CORE RULE

Operator interfaces are Function specializations where input and output types match; BiFunction accepts two arguments.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
BinaryOperator<Integer> sum = Integer::sum;
System.out.println(sum.apply(8, 5));
```

EXPECTED OUTPUT / RESULT

```
13
```

WHY THIS WORKS — DEEP EXPLANATION

UnaryOperator<T> is Function<T,T> and BinaryOperator<T> is BiFunction<T,T,T>. These are useful for updates, reductions, and merging map values. When combining two values into a different type, BiFunction<A,B,R> is a better match. Interface choice documents intent to future readers.

PRODUCTION DETAIL / SECOND-LAYER REASONING

BinaryOperator<T> has same-type operands and result, whereas BiFunction<A,B,R> can produce a third type. Map.merge uses BiFunction<V,V,V> to combine an existing value and an incoming value; a null result can remove the mapping. Keep combining operations deterministic when possible.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 13.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Why is a sum over Integer appropriately BinaryOperator<Integer>?

LESSON 11 OF 22

Primitive functional interfaces and boxing

THE CORE RULE

IntPredicate, IntFunction, IntUnaryOperator, ToIntFunction and related types can avoid unnecessary boxing in hot numeric paths.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
IntUnaryOperator twice = n -> n * 2;
System.out.println(twice.applyAsInt(6));
```

EXPECTED OUTPUT / RESULT

```
12
```

WHY THIS WORKS — DEEP EXPLANATION

Generic Function<Integer,Integer> may box/unbox int values. Primitive specializations make primitive intent obvious and often avoid allocations. Do not optimize prematurely: readability is more important for ordinary business loops. Choose the specialization that matches both argument and return type.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Primitive specializations vary by both input and output. IntFunction<R> receives int and returns R; ToIntFunction<T> receives T and returns int; IntUnaryOperator maps int to int. Select the exact method name such as applyAsInt(), getAsInt(), or test().

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 12.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Which interface maps String to int without boxing the result?

LESSON 12 OF 22

Method reference: static and bound instance

THE CORE RULE

A method reference is a concise alternative to a lambda that simply invokes an existing compatible method.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Function<String,Integer> parse = Integer::parseInt;
System.out.println(parse.apply("42"));
```

EXPECTED OUTPUT / RESULT

```
42
```

WHY THIS WORKS — DEEP EXPLANATION

Four common forms exist: Class::staticMethod, receiver::instanceMethod, Type::instanceMethod and Type::new. A reference must match the target functional interface signature. The expression before :: can be evaluated when the reference is created, while the method is called later.

PRODUCTION DETAIL / SECOND-LAYER REASONING

A bound method reference System.out::println captures a particular PrintStream receiver, while String::length waits for a String receiver argument. In review, expand the reference to its equivalent lambda to verify receiver and argument placement. Beware that referencing an overloaded method may require clear target types.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 42.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Rewrite Integer::parseInt as an equivalent lambda.

LESSON 13 OF 22

Method reference: unbound instance method

THE CORE RULE

Type::instanceMethod treats the first functional argument as the receiver; subsequent arguments become method arguments.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Function<String,Integer> length = String::length;
System.out.println(length.apply("Lambda"));
```

EXPECTED OUTPUT / RESULT

```
6
```

WHY THIS WORKS — DEEP EXPLANATION

String::length is a shorthand for s -> s.length(). For a method accepting arguments, e.g. String::startsWith, the functional interface needs a receiver and the argument. Bound references such as System.out::println instead capture a particular receiver. A method reference is not always clearer than a lambda with named parameters.

PRODUCTION DETAIL / SECOND-LAYER REASONING

For BiPredicate<String,String>, String::startsWith is compatible because the first String is receiver and second is the prefix. A method reference may look deceptively compact when there are overloads: changing the target functional interface can change which overload Java selects.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 6.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

What is the implicit receiver in String::length?

LESSON 14 OF 22

Constructor references and simple factories

THE CORE RULE

Type::new uses a constructor compatible with the functional interface signature and return type.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Supplier<ArrayList<String>> maker = ArrayList::new;
System.out.println(maker.get().size());
```

EXPECTED OUTPUT / RESULT

```
0
```

WHY THIS WORKS — DEEP EXPLANATION

Constructor references do not guarantee singleton behavior: this supplier creates a fresh list per call. Overloaded constructors are chosen by target typing. Array constructor references like String[]::new are also possible, but they are less central to everyday business code.

PRODUCTION DETAIL / SECOND-LAYER REASONING

ArrayList::new used as Supplier<ArrayList<String>> calls the no-arg constructor. Used as IntFunction<ArrayList<String>>, it may call the constructor taking an initial capacity. A constructor reference does not hide object creation cost and never automatically reuses objects.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 0.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Would maker.get() == maker.get() normally be true?

LESSON 15 OF 22

Effectively final captured local variables

THE CORE RULE

A lambda may capture a local variable only when it is final or effectively final; reassignment breaks eligibility.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
int bonus = 3;
IntUnaryOperator f = n -> n + bonus;
System.out.println(f.applyAsInt(7));
```

EXPECTED OUTPUT / RESULT

```
10
```

WHY THIS WORKS — DEEP EXPLANATION

Capture retains a value for the lambda to use, not a mutable local variable slot. Java prevents capturing reassigned locals, avoiding confusing closure semantics. Mutating the state of a referenced object is still possible; this does not make such mutation automatically safe, especially across threads. Use immutable inputs when practical.

PRODUCTION DETAIL / SECOND-LAYER REASONING

An effectively final local can be read but not reassigned after capture. The object it refers to might still change, so captured state is not automatically immutable. Under concurrency, visibility and atomicity still need proper synchronization or safe data structures. Prefer mapping immutable values in stream pipelines.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 10.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Would bonus++ after creating f compile? Why not?

LESSON 16 OF 22

Lambda this and scope versus anonymous classes

THE CORE RULE

Inside a lambda, this refers to the enclosing instance; it does not refer to a newly created lambda object.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
class Demo
{
String id = "outer";
Runnable r = () -> System.out.println(this.id);
}
```

EXPECTED OUTPUT / RESULT

```
Compiles
```

WHY THIS WORKS — DEEP EXPLANATION

Lambdas use lexical scoping: this and super preserve the meaning they have in the surrounding code. An anonymous inner class introduces its own this and may have different shadowing rules. Lambda parameters also cannot reuse local names already defined in the enclosing scope. These differences matter during refactoring from anonymous classes.

PRODUCTION DETAIL / SECOND-LAYER REASONING

A lambda is not an anonymous inner class with abbreviated spelling. Lexical this/super and variable-shadowing rules differ. Inside a static method, this is unavailable even in a lambda. Inside a class, a lambda can access private members of the enclosing instance as ordinary lexical code can.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: Compiles.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

What does this mean in a lambda written inside a static method?

LESSON 17 OF 22

Comparator.comparing and thenComparing

THE CORE RULE

Comparator lambdas and method references express ordered business rules without manual compareTo chains.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
List<String> names = new ArrayList<>(List.of("eve", "Ada"));
names.sort(String.CASE_INSENSITIVE_ORDER);
System.out.println(names);
```

EXPECTED OUTPUT / RESULT

```
[Ada, eve]
```

WHY THIS WORKS — DEEP EXPLANATION

Prefer Comparator.comparingInt for numeric keys when appropriate, and thenComparing for deterministic tie-breaking. Comparator rules must be transitive and consistent for sorting to behave correctly. A bad comparator based on subtraction can overflow. Know how nullsFirst/nullsLast explicitly handle nullable sort fields.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Comparator.comparingInt avoids boxing an int sort key. thenComparing prevents ties from relying accidentally on original order. Subtracting integers in a comparator, such as a.age()-b.age(), can overflow and violate comparison contracts: use Integer.compare or Comparator.comparingInt.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: [Ada, eve].

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Sort people by age ascending, then name ascending.

LESSON 18 OF 22

Function chaining with Optional and collections

THE CORE RULE

Functional interfaces are used by forEach, removeIf, computeIfAbsent, and Optional methods; learn the contracts rather than just their syntax.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
List<String> words = new ArrayList<>(List.of("a", "bb"));
words.removeIf(w -> w.length() == 1);
System.out.println(words);
```

EXPECTED OUTPUT / RESULT

```
[bb]
```

WHY THIS WORKS — DEEP EXPLANATION

Collection methods often accept Predicate, Consumer or Function. A removal predicate should not mutate the same collection structurally outside the method contract. computeIfAbsent may skip its function entirely when a non-null existing mapping is present. Optional.map invokes mapping only when a value is present.

PRODUCTION DETAIL / SECOND-LAYER REASONING

removeIf changes the receiver collection through its documented operation. list.forEach() and stream.forEach() have different processing semantics when ordering or parallelism is involved. Be careful not to modify a collection structurally while iterating over it through an incompatible pathway.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: [bb].

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Does computeIfAbsent call the mapper when a non-null value exists?

LESSON 19 OF 22

Exceptions inside lambda bodies

THE CORE RULE

The checked-exception rules come from the abstract method of the target interface, not from the lambda keyword.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Function<String,Integer> parse = Integer::parseInt;
System.out.println(parse.apply("12"));
```

EXPECTED OUTPUT / RESULT

```
12
```

WHY THIS WORKS — DEEP EXPLANATION

Function.apply cannot declare IOException, so a lambda using an IO method must handle it or translate it to an unchecked exception, ideally preserving its cause. Do not silently swallow checked exceptions or wrap every error without context. A custom functional interface declaring throws can be justified at controlled boundaries.

PRODUCTION DETAIL / SECOND-LAYER REASONING

An ordinary Function<String,String> cannot propagate a checked IOException in apply(). You can catch it, wrap it in UncheckedIOException (appropriate for I/O), and retain the cause. Catching Exception around every callback is not a sound recovery strategy; choose the handling boundary deliberately.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 12.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Why can a Runnable lambda not freely throw a checked IOException?

LESSON 20 OF 22

Deferred evaluation, side effects and debugging

THE CORE RULE

Lambdas are executed when an operation invokes the interface method; defining a lambda alone normally does not run its body.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
Runnable r = () -> System.out.println("later");
System.out.println("now");
r.run();
```

EXPECTED OUTPUT / RESULT

```
now
later
```

WHY THIS WORKS — DEEP EXPLANATION

Distinguish creation of behavior from invocation. When debugging, place breakpoints inside the lambda and in the caller that invokes it. Avoid relying on identity, generated class names or toString output. Be deliberate about captured mutable state and parallel execution; the callback may run on a different thread depending on the API.

PRODUCTION DETAIL / SECOND-LAYER REASONING

The invocation time of a callback is an API contract question. An eager List.forEach invokes immediately, while a Stream.filter intermediate operation defers execution until a terminal operation. Callback code may be invoked zero, one or multiple times depending on the caller. Never rely on hidden side effects for correctness.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: now / later.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Where would you put breakpoints to observe when this Runnable executes?

LESSON 21 OF 22

Standard functional interfaces versus custom ones

THE CORE RULE

Prefer built-in interfaces when their name and contract communicate intent; create custom interfaces for domain semantics or checked contracts.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
@FunctionalInterface
interface Rule
{
boolean allows(String action);
}
```

EXPECTED OUTPUT / RESULT

```
Compiles
```

WHY THIS WORKS — DEEP EXPLANATION

A meaningful domain interface can improve API readability and permit a carefully designed default method. Avoid inventing single-method interfaces whose signatures exactly duplicate standard Java types without conveying useful domain meaning. The @FunctionalInterface annotation helps protect your intention during maintenance.

PRODUCTION DETAIL / SECOND-LAYER REASONING

A custom domain interface named AuthorizationRule may better document a permission decision than Predicate<String>; still, avoid creating unnecessary bespoke interfaces for every one-line callback. Function naming, input/output typing, and checked failure semantics should all be meaningful.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: Compiles.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

Would Predicate<String> be sufficient for a generic string test?

LESSON 22 OF 22

Performance, correctness and when not to use lambdas

THE CORE RULE

Prefer lambdas for short, clear behavior; use named methods/classes for complex state, branching, lifecycle or debugging needs.

JAVA EXAMPLE — ALLMAN STYLE WHEN BLOCKS ARE USED

```
List<String> words = List.of("a", "bb");
long count = words.stream().filter(w -> w.length() > 1).count();
System.out.println(count);
```

EXPECTED OUTPUT / RESULT

```
1
```

WHY THIS WORKS — DEEP EXPLANATION

Lambda syntax alone does not guarantee better performance or automatic parallelism. Streams can make transformations concise but add a layer of execution semantics covered next chapter. A method reference can reduce clutter; a named function may communicate domain meaning better. Avoid using side-effect-heavy lambdas as hidden control flow.

PRODUCTION DETAIL / SECOND-LAYER REASONING

Lambdas should reduce accidental complexity, not become a puzzle. Three or four nested callbacks with logging, mutation and exception wrappers can be harder to maintain than a named method and explicit loop. Profile performance before replacing readable code with specialized functional machinery.

TRACE THE BEHAVIOR

1. Identify the target functional interface and its abstract method. 2. Read the lambda parameters, captured state and body. 3. Find the call to test(), apply(), accept(), get() or the relevant callback method. 4. Only then predict the visible result: 1.

COMMON PITFALL

Mistaking the lambda declaration for an invocation, or assuming that a method reference works without a compatible target interface. For stateful callbacks, also ask who owns the state and on which thread the callback may execute.

APPLY THE RULE

When would an ordinary loop be clearer than chained callbacks?

Quick reference: choose the interface

Predicate<T> — T → boolean; test; filter / validate

Function<T,R> — T → R; apply; transform values

Consumer<T> — T → void; accept; perform an effect

Supplier<T> — () → T; get; deferred value / factory

UnaryOperator<T> — T → T; apply; same-type transform

BinaryOperator<T> — (T,T) → T; apply; combine equal types

ToIntFunction<T> — T → int; applyAsInt; avoid boxing output

Practical checklist

Does the lambda have a clear target type? Is the behavior short enough to remain readable? Are captures effectively final and shared-state effects safe? Is exception translation preserving the cause? Is a method reference genuinely clearer? Do you understand exactly when the callback runs?

Runnable companion project

Open the Maven project in IntelliJ. Run six demo classes independently. The project targets Java 17 and includes exact expected console-output text files. Code uses Allman-style braces for blocks.